

JUNIOR SYSTEM ADMINISTRATOR PROTOCOL — BEGINNER TRACK

MODULE 1: HARDWARE ASSEMBLY & YOUR ACCOUNT

Objective: Put the workstation together and create your administrator account.

Tech Briefing

A computer is a circuit of parts that need each other. Power comes in through the USB-C cable. The "brain" (the processor) runs programs stored on the MicroSD card. The keyboard and mouse send signals *in*; the monitor shows results *out*. Once the parts are connected, the operating system asks you to create a **user account** — your identity on the machine. Your account will be an **administrator**, which means it has permission to change important settings.

Main Mission

1. **Physical Assembly:** Connect the micro-HDMI cable to the port labeled **HDMI0** on the Pi, then connect the other end to the monitor. Plug the USB keyboard and mouse into the ports. Do not apply power yet.
2. **Boot Initialization:** Insert the pre-flashed MicroSD card into the slot on the bottom of the Pi. Connect the power adapter to the wall, then plug it into the Pi.
3. **Account Provisioning:** Follow the on-screen wizard to create your primary user account. Choose a unique username and a secure password.
4. **Network Integration:** Use the desktop taskbar icon to connect the device to the family Wi-Fi network.

Side Quest

Find the application named **Terminal** in your menus and click it to open. The Terminal lets you talk to the computer by typing commands instead of clicking.

- Type `whoami` and hit Enter. The computer answers with the name of the account that is currently logged in. Check that it matches the username you just created.
- Type `ping google.com` and hit Enter. Your Pi is sending tiny "are you there?" messages to Google and timing the replies. After a few lines of output, press `Ctrl-C` to stop it. About how many **ms** (milliseconds) did each reply take?

Debrief

Tell Dad in your own words: what does `whoami` do, and what was `ping` actually measuring?

MODULE 2: FILE SYSTEMS & NETWORK PRINTING

Objective: Learn where files live, and print a document over the network.

Warm-Up Drill (no peeking!)

From memory: open the Terminal and run the command that shows which account is logged in. Then run the command that tests whether the internet is reachable (and remember how to stop it).

Tech Briefing

Linux stores every file in one giant upside-down tree. The tree starts at the **root**, written as just `/`. Your personal stuff lives in your **Home** folder, at `/home/yourusername/`. An **absolute path** is the full address of a file, starting all the way from `/` — like a complete street address for the computer. Example: `/home/sam/Documents/homework.txt`.

To print, Linux uses a system called **CUPS** that searches the local network for printers.

Main Mission

1. **Directory Generation:** Open the File Manager utility. Navigate to your Home directory, open the **Documents** folder, and create a new folder named **System_Logs**.
2. **Document Creation:** Open the **Text Editor**. Write an inventory listing every physical hardware component connected to your Pi (brand of mouse, size of monitor, keyboard type).
3. **Structured Save:** Save this document as `hardware_inventory.txt` directly inside your new **System_Logs** folder.
4. **Peripheral Mapping:** Open the system settings menu and select **Print Settings** (or **Printers**). Click "Add", select "Network Printer", and wait for the system to discover the family printer on the network.
5. **Hardcopy Execution:** Open your `hardware_inventory.txt` file and print a physical copy to confirm the driver and network path are operational.

On Your Own (no recipe this time)

Create a second folder inside **Documents** named **Reports**, and save a text file inside it named `test_report.txt` containing one sentence about how the printing went. You just did all of these steps — now do them without instructions.

Side Quest

In the File Manager, go into your **System_Logs** folder. Click on the path bar at the top of the window to change it from icons to text. Copy or write down that exact absolute path string.

Debrief

Tell Dad in your own words: what is an *absolute path*? Say the absolute path to your **System_Logs** folder out loud, starting with `/`.

MODULE 3: PASSWORDS & THE WEB

Objective: Set up a password manager and safely download an image from the web.

Warm-Up Drill (no peeking!)

From memory: without opening the File Manager, say the absolute path to your **System_Logs** folder out loud, starting with /. Then open the File Manager and check the path bar — were you right?

Tech Briefing

A web browser downloads code from other computers and turns it into the pages you see. Because you'll need passwords for many websites, professionals use a **password manager** — a locked vault app that remembers strong passwords so you don't have to. Ours is called **Bitwarden**. Logging into the browser also turns on safe-search filters.

Main Mission

1. **Browser Initialization:** Open the **Chromium** web browser from your desktop taskbar.
2. **Extension Deployment:** Navigate to the Chrome Web Store, search for **Bitwarden**, and install the extension.
3. **Identity Authentication:** Click the Bitwarden extension icon in the top right and log in with your credentials.
4. **Google Login:** Navigate to **google.com** and sign in to your account. Bitwarden should pop up and offer your saved credentials.
5. **Targeted Research:** Use Google to find the official wiki page for your favorite video game. Locate a high-resolution image of a character on that wiki.
6. **Disk Write:** Right-click the image, select "Save Image As...", and save it into your **Pictures** folder.

On Your Own (no recipe this time)

Find and save **one more image** — this time of your favorite animal — but save it into your **Documents/Reports** folder from Module 2 instead of **Pictures**. Then find it in the File Manager to prove it landed in the right place.

Side Quest

Set the downloaded character image as your system desktop background. Right-click the desktop, open **Desktop Preferences**, and point it at the exact folder where your image was saved.

Debrief

Tell Dad in your own words: why is a password manager safer than using the same password everywhere?

MODULE 4: COPY, CUT, AND THE TRASH

Objective: Master copying, moving, and safely deleting files.

Warm-Up Drill (no peeking!)

From memory: create a new folder inside **Documents** called **Warmup_4**, and save a text file into it from the Text Editor. Two modules of practice — you've got this.

Tech Briefing

There are two ways to put a file somewhere new. **Copy** makes a duplicate — now there are two files. **Cut** (also called *move*) picks the file up from its old spot and puts it in the new one — still just one file. Deleting a file normally doesn't destroy it right away: it goes to the **Trash**, a holding area, until you *empty* the Trash. That's your safety net.

Main Mission

1. **The Duplicate Target:** Open your File Manager and go to your **Pictures** folder. Locate the character image you saved during Module 3.
2. **The Copy Sequence:** Right-click the image and select **Copy** (or press **Ctrl + C**). Navigate to your **Documents** folder, right-click an empty space, and select **Paste** (**Ctrl + V**). Verify that you now have two identical copies of this image in different folders.
3. **The Cut Sequence:** Go back to your **Pictures** folder. Create a new folder inside it called **Wallpapers**. Right-click your original image, select **Cut** (**Ctrl + X**), open the **Wallpapers** folder, and **Paste** it inside. Check your main Pictures folder to confirm the original image disappeared from there.
4. **The Disposal:** Go to the **Documents** folder where you pasted the duplicate image. Right-click that duplicate file and select **Move to Trash** (or press the **Delete** key).
5. **Purging the System:** Find the **Trash** icon on your desktop or side panel. Open it to verify your file is waiting inside. Right-click the Trash icon and select **Empty Trash** to permanently clear the storage space.

On Your Own (no recipe this time)

Do a full round trip with no instructions: **copy** your **hardware_inventory.txt** from **System_Logs** onto the Desktop, then **move** (cut) that Desktop copy into your **Warmup_4** folder, then send it to the **Trash** and empty the Trash. Say out loud which step was a copy and which was a move.

Side Quest

Create a blank text document called **temp.txt** and save it to your desktop. Highlight it and press **Shift + Delete**. Now go look in your Trash folder — and explain to Dad why the file didn't show up in the Trash this time.

Debrief

Tell Dad in your own words: after a **copy**, how many files exist? After a **cut**? When is a file truly gone forever?

MODULE 5: HIDDEN FILES & GAME SAVES

Objective: Install software with Pi-Apps, find hidden folders, make your own hidden folder, and back up a Minecraft world.

Warm-Up Drill (no peeking!)

From memory: copy any image from **Pictures/Wallpapers** into **Documents**, then send the copy to the Trash and empty it. Narrate copy vs. move while you do it.

Tech Briefing

Games save your worlds and settings as regular files on the disk. To keep you from deleting them by accident, Linux **hides** those folders. The hiding rule is simple: **any file or folder whose name starts with a dot (.) is hidden**. That's the whole trick — **.minecraft-pi** is hidden, **minecraft-pi** would not be. Hidden files are still there; the File Manager just skips them unless you ask to see them.

Not every app ships with Raspberry Pi OS. A community tool called **Pi-Apps** works like an app store: pick a program and it installs it for you.

Main Mission

1. **Install Pi-Apps:** Open the Terminal and run:

```
wget -q0- https://raw.githubusercontent.com/Botspot/pi-apps/master/install | b
```

When it finishes, open Pi-Apps from the application menu (**Accessories** → **Pi-Apps**) or by typing **pi-apps** in the terminal.

2. **Install Minecraft Pi (Modded):** In Pi-Apps, open the **Games** category, select **Minecraft Pi (Modded)**, and click **Install**. Wait until the install completes.
3. **Play and Build:** Launch Minecraft Pi (Modded) from the Games menu. Create or open a world, build a recognizable structure (like a tall tower), then quit the game completely so it saves to disk.
4. **Hidden Directory Hunt:** Open the File Manager and go to your Home directory. Turn on **Show Hidden** (under the View menu, or press **Ctrl+H**) so folders that start with a dot appear. Press **Ctrl+H** a few times and watch the dot-folders blink in and out of view.
5. **Build Your Own Hideout:** Now *make* a hidden folder yourself. In the Terminal, type:

```
mkdir /home/YOURUSERNAME/.hideout
```

(Use your real username.) Go back to the File Manager: with hidden files OFF it's invisible; press **Ctrl+H** and there it is. The **dot at the front of the name** is the only thing making it hidden.

6. **World Extraction:** Find the folder named **.minecraft-pi**. Inside it, locate your world data and copy that world folder into your **Documents** folder as a backup.
7. **Verification Test:** Relaunch the game, enter your world, and destroy your structure. Quit the game. Copy your backup from **Documents** back over the ruined world folder inside **.minecraft-pi**. Launch the game again and confirm your structure is restored.

On Your Own (no recipe this time)

Make a hidden folder in your Home directory named **.top_secret**, and save a text file inside it with a secret message for Dad. Then show him how to reveal it in the File Manager. No commands provided — you just did this in step 5.

Side Quest

Still inside `.minecraft-pi`, open `options.txt` in the Text Editor. Look at how the game stores settings (like volume or controls) as plain text you can read and edit.

Debrief

Tell Dad in your own words: what exactly makes a file hidden in Linux? (Hint: it's one character.) Why would a game *want* its save folder hidden?

MODULE 6: THE COMMAND HELP SYSTEM

Objective: Learn how commands can teach you their own tricks, using flags and built-in manuals.

Warm-Up Drill (no peeking!)

From memory: in the Terminal, make a hidden folder in your Home directory called `.warmup6`. Then prove it exists in the File Manager. What made it hidden?

Tech Briefing

Here's a secret: you don't have to memorize what every command can do — **commands can tell you themselves**. You ask by adding a **flag** after the command name. Flags start with dashes, like `-h` or `--help`. Some flags ask for help; other flags change how the command behaves. This module is the most important one in the whole track, because it teaches you how to *figure things out* instead of memorizing them.

Main Mission

1. **Terminal Help Probe:** Open your Terminal. Type `mkdir --help` and press Enter. Read the output. Find the section that explains what the `-p` flag does.
2. **Directory Trees:** Use what you just learned. Instead of making one folder at a time, use that flag to build a whole nested chain of folders with one single command: `mkdir -p ~/Downloads/Games/Retro/Python/Sources`. (The `~` is a shortcut that means "my Home folder.")
3. **Where Am I?:** Type `pwd` and press Enter. It prints the absolute path of the folder your Terminal is currently standing in — "print working directory." Now type `cd ~/Downloads` (change directory), then `pwd` again. Watch the path change.
4. **The List Helper:** Type `ls --help`. Look through the options until you find the flag that lists files in a "long" format showing file sizes and dates. Run `ls` with that flag inside your Downloads folder to test it.
5. **The Disk Space Analyzer:** Type `df` and press Enter. The numbers are in raw blocks — confusing. Type `df --help` and find the flag that makes the output "human readable" (gigabytes and megabytes). Run it with that flag.

On Your Own (no recipe this time)

Remember hidden files from Module 5? The plain `ls` command skips them, just like the File Manager does. Use `ls --help` to discover the flag that shows **all** files, including hidden ones. Then run it in your Home folder and find `.hideout` and `.top_secret` in the list. Nobody tells you the flag — the help text is your map.

Side Quest

Type `man mkdir` into the terminal. This opens the complete "Manual Page" book for that tool. Use your arrow keys to scroll, and press `q` to quit back to your prompt.

Debrief

Tell Dad in your own words: what is a *flag*? If you met a brand-new command tomorrow, what would you type to learn what it can do?

MODULE 7: DOWNLOADING CODE & GORILLAS PHYSICS

Objective: Grab a raw code file from the web with the Terminal, then modify the game's physics.

Warm-Up Drill (no peeking!)

From memory: use the one-command trick from Module 6 to build this whole folder chain at once:

`~/Missions/Practice/Run1`. Then use the long-format `ls` flag to prove `Missions` exists. Stuck? Re-read Module 6, close it, and try again from memory.

Tech Briefing

To download files in the Terminal, we use `wget` ("web get"). You give `wget` a web link, and it saves that file into the folder your Terminal is standing in.

One trap: a normal GitHub page is a *decorated webpage* — if you `wget` it, you get the decoration, not the code. To get the pure code, you need the **Raw** link. Today you'll download a Python remake of *Gorillas* — a 1990 MS-DOS game where gorillas throw exploding bananas over a city skyline.

Main Mission

1. **Targeting the Directory:** Open your Terminal and move into the retro game folder you built during Module 6. You know the two commands for "change folder" and "prove where I am" — use them both. (The path is `~/Downloads/Games/Retro/Python/Sources`.)
2. **Navigating the Code Vault:** Open Chromium and go to `github.com`. In the search bar, type `ByScreams/Gorillas-Python` and press Enter. Click the matching project.
3. **Locating the Main Script:** In the list of project files, find `gorillas.py` and click it. You'll see a preview of the code.
4. **Extracting the Raw Link:** Above the code box, find and click the button labeled **Raw**. Notice the page turns into plain text on a white background — that's the pure code with no decoration.
5. **Executing Web Get:** Copy the entire URL from the address bar (it should start with `https://raw.githubusercontent.com/...`). In your Terminal, type `wget` , paste the URL after the space, and press Enter.
6. **Execution & Modification:** When the download hits 100%, launch the game with `python3 gorillas.py`. Play a round, then close the game window. Open the file with `nano gorillas.py`. Find the constant labeled `GRAVITY = 9.8` (or `SPEEDCONST`). Change the number to `2.0` for moon gravity. Save with `Ctrl + O`, Enter, exit with `Ctrl + X`. Run the game again to test your custom build.

On Your Own (no recipe this time)

Change the gravity again — this time make it *stronger* than Earth (try `20`). Open the file, edit, save, exit, and run it, all from memory. Which planet does it feel like now?

Side Quest

Re-open the script in `nano`. Find where the game sets player names and change "Player 1" to your own gaming handle.

Debrief

Tell Dad in your own words: what's the difference between a normal GitHub page and the **Raw** page, and why did `wget` need the raw one?

MODULE 8: PROJECT REPOSITORIES & STORY GAMES

Objective: Clone an entire project from GitHub with `git`, then run and edit Python adventure games.

Warm-Up Drill (no peeking!)

From memory: run the command that shows disk space in human-readable form. Then run the command that prints which folder your Terminal is standing in.

Tech Briefing

Last module you downloaded *one file* with `wget`. But real software projects are whole folders full of files that need each other. Developers bundle a project into a **repository** ("repo") on GitHub. The tool `git` can copy an entire repository to your machine with one subcommand: `git clone`. Think of it as `wget` for a whole project at once.

Main Mission

1. **Setting the Stage:** In your Terminal, create a folder called `Games` in your Home directory (the Module 6 flag means it's fine if it already exists), then move into it. You know both commands — no copy-paste this time. Check yourself with `pwd`: it should print `/home/yourusername/Games`.
2. **Finding the Repository Target:** In Chromium, go to `github.com` and search for `junior-system-admin` by the user `jermatic1`. Click the project. (Shortcut if search is slow: `https://github.com/jermatic1/junior-system-admin`.)
3. **Acquiring the Project Link:** On the repository page, click the bright green **Code** button. Under the "Local" tab, select **HTTPS**, and click the small square icon to copy the URL. It should look like: `https://github.com/jermatic1/junior-system-admin.git`
4. **Cloning the Repository:** Back in your Terminal (still inside `~/Games`), type `git clone` (with a space) and paste the URL. Press Enter and watch it download the whole project.
5. **Enter the Games Bay:** Move into the cloned project's `games` folder and list what's inside — `cd` and `ls` are old friends by now. You should see `iron_gates.py`, `starship_traveller.py`, and `engine.py`.
6. **Launch Iron Gates:** Run the first adventure:

```
python3 iron_gates.py
```

Play through to an ending. Use the number keys to choose your path. Press **Ctrl+C** to exit early if needed.

- If Python complains it cannot find a module named `rich`, install it with `sudo apt install python3-rich -y` and run the game again.
7. **Inspect the Source:** Open the game with `nano iron_gates.py`. Find the big `STORY = { ... }` dictionary near the top. Notice how each scene has a title, some text, and numbered choices pointing to the next scene.
 8. **Modify the Adventure:** Change something in the story! Ideas: rewrite a scene in your own words, add a new choice leading to a new scene, or rename the victory message. Save, exit, and play your custom version.

On Your Own (no recipe this time)

Launch the second adventure, `starship_traveller.py`, and play a full run (watch your hull and fuel). Then open it in the editor and change one thing — all from memory.

Side Quest

With both games open in your mind: **what looks the same** between `iron_gates.py` and `starship_traveller.py`? What did the authors put in a shared file (`engine.py`) so both games could reuse it?

Debrief

Tell Dad in your own words: when would you use `wget`, and when would you use `git clone`?

MODULE 9: FULL BEGINNER REVIEW — MISSION CONTROL FIELD TEST

Objective: Prove you can do everything from Modules 1–8 with a goal and no recipe.

Tech Briefing

Real admins don't get step-by-step instructions. They get a **goal** and combine the tools they already know. Every task below is something you have already done in Modules 1–8. There are **no commands printed in the missions this time** — that's on purpose.

Rules of engagement:

- Try each task from memory first.
- Stuck? Use the numbered **Hints** at the end of this module. Each hint tells you *which module* the skill came from and gives a nudge — not the answer.
- Keep a tally of how many hints you use. Check your rank at the end. Using hints is not cheating — it's how real admins use documentation.

Field Test

1. **Radio Check:** Open the Terminal. Run the command that shows who is logged in, and the command that checks the internet is reachable (remember how to stop it).
2. **Build the Base:** Using a single command *per folder* (or fewer, if you know a trick), build this tree in your Home directory, then prove it exists with a long-format listing:
 - `Mission_Control` containing three folders: `logs`, `media`, `archive`
3. **Write the Briefing:** In the Text Editor, create `briefing.txt` and save it inside `Mission_Control/logs`. It must contain your name, today's date, and the **absolute path** of the `Mission_Control` folder. Get the path from the Terminal — there's a command that prints exactly where you're standing.
4. **Hardcopy:** Print `briefing.txt` on the family printer.
5. **Asset Recovery:** In Chromium, find a cool **SpaceX** image and save it into `Mission_Control/media`. Then set it as your desktop wallpaper.
6. **Copy, Cut, Trash:** Copy the SpaceX image from `media` into `archive`. Make one extra duplicate anywhere under `Mission_Control`, then *move* (cut) it into `archive`. Send one spare to the Trash, confirm it's sitting in the Trash, then empty the Trash. Narrate which operations were copies and which were moves.
7. **The Hidden Cache:** From the Terminal, create a hidden folder in your Home directory called `.mission_notes`, and put a text file inside it. Then reveal it in the File Manager, and *also* list it in the Terminal using the `ls` flag that shows hidden files.
8. **Disk Report:** Run the command that reports disk space in human-readable units. Find the line for your main disk (`/`). Add the free-space number as a new line in `briefing.txt` and save it.
9. **Teach Yourself Something:** Use `--help` on the `ls` command to discover a flag you have *never used before* — for example, one that sorts newest-first. Run it in `Mission_Control/media` and tell Dad what it did. This task is the whole point of Module 6: commands can teach you their own tricks.
10. **The Games Bay Returns:** In the Terminal, navigate to the story games you cloned in Module 8 and launch either adventure. Reach an ending. Then open its code in the terminal editor, change one line of story text, save, and play your changed version.

Side Quest: The 60-Second Tour

Give Dad a guided tour, narrating every click: start at the SpaceX wallpaper, open `Mission_Control`, show `briefing.txt` and the image in `media`, reveal the hidden `.mission_notes` folder, and finish by showing the free space on `/` in the Terminal.

Scoring

Count your hints:

- **0–2 hints:**  **Junior System Administrator.** Field-certified. You're ready for the Advanced Track (Modules 10+).

- **3–5 hints:**  **Field Technician.** Solid. Redo just the tasks where you needed hints, in a few days, from memory.
- **6+ hints:**  **Cadet.** No problem — this test is *hard* with no recipes. Rerun the Warm-Up Drills from Modules 2–8 this week, then take the field test again. Passing on the second try counts just as much.



Hints (use freely — tally each one)

1. **Radio Check** — Module 1's Side Quest. One command asks "who am I?", the other sends "are you there?" messages to Google. `Ctrl-C` stops a running command.
2. **Build the Base** — Module 6. The folder-making command has a flag that builds a whole nested chain (and never complains if a folder already exists). `~` means your Home folder. Run it once per subfolder if you like: `logs`, `media`, `archive` each live *inside* `Mission_Control`.
3. **Long listing** — Module 6. You found this flag by reading `ls --help`. It's a single lowercase letter.
4. **The path for the briefing** — Module 6, step 3. Change directory into `Mission_Control` first, then run the command that prints your working directory.
5. **Printing** — Module 2. Open the file in the Text Editor and look in its menus for Print; the family printer was already set up.
6. **Saving an image** — Module 3. Right-click the image in the browser. When the save window opens, steer it to the right folder before clicking Save.
7. **Wallpaper** — Module 3's Side Quest. Right-click the desktop.
8. **Copy vs. Cut** — Module 4. Copy leaves the original behind (`Ctrl+C`/`Ctrl+V`); Cut removes it (`Ctrl+X`/`Ctrl+V`). Trash lives on the desktop or side panel; "empty" is on its right-click menu.
9. **Hidden folder** — Module 5, step 5. One character at the front of a name is the entire hiding trick. `Ctrl+H` toggles them in the File Manager.
10. **Hidden files in the Terminal** — Module 6's On Your Own. You discovered this `ls` flag yourself using `--help`. It's the first letter of "all."
11. **Disk space** — Module 6, step 5. The command's name is two letters ("disk free"), plus the human-readable flag.
12. **Games Bay** — Module 8. The project lives in `~/Games/junior-system-admin/games`. Change directory there, list the files, run one with `python3`, and edit with `nano`. Save is `Ctrl+O` then Enter; exit is `Ctrl+X`.